Università degli Studi di Trieste

Dipartimento di Matematica, Informatica e Geoscienze - MIGE

Corso di Laurea in Intelligenza Artificiale e Data Analytics

Tesi di laurea triennale

Hybridizing Deep Learning and Classical Mathematical Modeling: theory and new applications

Laureando:
Dino Meng
Matricola: SM3201466

Relatore:
Prof. Alberto d'Onofrio

ANNO ACCADEMICO 2025–2026

Dino, inserisci qui la dedica che vuoi

«I do not know what I may appear to the world;
but to myself I seem to have been only like a boy,
playing on the sea-shore, and diverting myself,
in now and then finding a smoother pebble
or a prettier shell than ordinary,
whilst the great ocean of truth
lay all undiscovered before me.»

Isaac Newton

«All models are wrong, but some are useful.»

George Box

Contents

Contents **ii**

List of Figures **iii**

List of Tables **iv**

Introduction **2**

1 Literature Review **5**
1.1 Physics-Informed Machine Learning 5
1.1.1 Universal Differential Equations 7
1.2 Symbolic Regression with SINDy 9
1.3 Some Mathematical Models 10
1.3.1 Population Biology 11
1.3.2 Neurons 13
1.3.3 Chua's Circuit 16

2 Methodology and Reproduction of the Main Results **18**
2.1 The Spruce Budworm Model 18
2.2 Modeling Generalised Logistic Growths 20
2.3 Lotka-Volterra Predator-Prey Equations 27

3 Advanced Case Study I: Neuron Modeling **36**
3.1 Single Firing-Rate Neurons 36
3.2 Network of Firing-Rate Neurons 36

4 Advanced Case Study II: Chua's Circuit Model **37**
4.1 ??? 37

5 Concluding Remarks **38**

List of Figures

1.1.1 The spectrum of scenarios of data and physics availability. This figure is directly taken from [18] 6
1.1.2 Three pathways to achieve a phyiscs-informed learning algorithm. This figure is directly taken from [18] 6
1.3.3 Bifurcation diagram of the Budworm model (eq. 1.15) for $\alpha = 0.02$ 13
1.3.4 Periodic Solution of Lotka-Volterra model (eq. 1.17) . . 14
1.3.5 Two steady states of equation 1.24 16
1.3.6 Instability of the homogeneous steady state under large J of equation 1.24 17

2.1.1 Budworm Model Training Data 19
2.1.2 Budworm UDE Results 20
2.2.3 Generalized Logistic First Experiment Results 22
2.2.4 Generalized Logistic First Experiment Results (SINDy) 22
2.2.5 Phase diagram of the generalized logistic model 24
2.2.6 Generated trajectories for different initial points 25
2.2.7 Results of second experiment, for $x_0 \in [0.01, 2]$ 25
2.2.8 Results of second experiment, for $n = 3, \ldots, 29$ 26
2.2.9 Third experiment: SINDy models 27
2.2.10 Comparison of third experiment's models 28
2.3.11 Simulation of the UDE-enhanced SINDy recovered model 30
2.3.12 Simulation of the UDE-enhanced SINDy recovered model, with post-structure identification parameter estimation 30
2.3.13 Birth and Death Rates of the wrongly inferred model . 31
2.3.14 Comparison of the alternative UDE architecture with the previously proposed one 32
2.3.15 Simulation of the recovered equations, case 1 ($c = 0.5$) 34
2.3.16 Simulation of the recovered equations, case 2 ($c = 2$) . 35

List of Tables

2.2.1 Model comparison metrics 23
2.2.2 Recovered coefficients 23
2.3.3 Recovered coefficients 29
2.3.4 Recovered coefficients for $c = 0.5$ 34
2.3.5 Recovered coefficients for $c = 2$ 34

List of Main Acronyms

UDE Univeral Differential Equation

ODE Ordinary Differential Equation

UODE Universal Ordinary Differential Equation

PDE Partial Differential Equation

ML Machine Learning

ANN Artificial Neural Network

ReLU Rectified Linear Unit

SiLU Sigmoid Linear Unit

RK4 Runge–Kutta Algorithm of 4th Order

RMSE Root Mean Square Error

SA Simulated Annealing

Introduction

MACHINE AND DEEP LEARNING Machine Learning is the field of study concerned with studying algorithms designed to discover hidden patterns in data and extrapolate them, possibly creating a model that is able to generalize the learned pattern on unseen data ([13]).
One of the most important class of algorithms taken into consideration are the so-called Artificial Neural Networks (ANN), which has origins with F. Rosenblatt's perceptron model in 1958 ([1]), in an attempt to create a mathematical representation of learning in biological systems. We call Deep Learning the subset of Machine Learning that deals with such models.
Cybenko's Universal Approximation theorem ([3]) motivates said subfield, as it implies that any sufficiently regular function can be approximated by a Neural Network with sigmoidal activation functions. So, there are a wide number of practical applications to specific cases, such as regression, classification, image recognition and segmentation, object detection, et cetera...
TODO: this part can be improved by integrating the other ML book (?)

DYNAMICAL SYSTEMS In applied mathematics, dynamical systems are systems that evolve in time and are applied to study the behaviour of some systems in sciences or engineering. In fact, dynamics was born as a branch of physics with Newton's conception of differential equations (DEs) ([12]). For example, Newton's second law of motion is a second-order ODE:

$$F(x(t), t) = m\frac{\mathrm{d}^2 x}{\mathrm{d}t^2}(t)$$

Mathematically, there are two ways to describe a dynamical systems: DEs and recurrence relations. We will only study DEs in this

thesis, although the other case can exhibit interesting behaviours.

HYBRID MODELS However, the two subject matters mentioned above are not completely independent from each other; rather, they can be joined to form more robust predictive models, or conversely, obtain meaningful insights about dynamical systems for mathematical modelling.
Physics-informed machine learning is the field that concerns itself with integrating mathematical models into ML learning algorithms ([18]). As mentioned in [18], there are many ways to go about this: some of these approaches include Physics-Informed Neural Networks ([14]), Neural Ordinary Differential Equations ([11]) and Universal Differential Equations ([16]).
TODO: improve the parts before this, maybe remove something? especially the

THESIS OVERVIEW In this thesis we will mainly discuss the Universal Differential Equations (UDEs) and its application to both ML and mathematical modeling.
Chapter 1 consists of a literature review of various articles and textbooks which are necessary to provide and understand the necessary methodologies and theory behind the experiments that will follow.
Then in Chapter 2 we will convey the essence of the UDEs, starting from a basic idea of the methodology and applying it to study a variant of the logistic model ([2]). Successively, we will apply more advanced methodologies to study the generalized logistic model and to reproduce the Lotka-Volterra experiment of [16] with some slight modifications.
As soon as we have a clear grasp on the essence of the UDEs, we will dedicate the next chapters in applying this methodology to more advanced cases.
In our first advanced case study in Chapter 3, we will apply UDEs to neuron modeling as detailed in [19] and in [8]. Particularly, we will focus on the capabilities of UDEs of extrapolating dynamics from a limited amount of data and reconstructing the parameters of the dynamics, such as the weight adjacency matrix.
The next and final (?) advanced case study in Chapter 4 will be about chaotic systems, considering in particular Chua's circuit

model; analogously, we will leverage UDEs in order to learn the hidden nonlinear terms of the differential equation, see how "well" it can recover the dynamics of a chaotic model through symbolic regression and how we can quantify the "good-fitness" of such model.
TODO: AGGIUNGERE DELLE CITAZIONI
Chapter 5 will conclude this work with some final remarks on the topic, discussing the limits and further potentialities of UDEs.
The code used to carry out the various experiments of this thesis is available in the following GitHub repository:

https://github.com/OdinMeng/Hybridizing-DeepLearning-and-ClassicalMathematicalModeling

N.B. il repository è ancora privato, lo pubblicizzerò dopo

Chapter 1

Literature Review

In this chapter we will have a general overview on the topics necessary to understand the work performed in the thesis.

1.1 Physics-Informed Machine Learning

Problems in ML and classical mathematical modeling represent two different scenarios in terms of availability in data and physics knowledge: with Machine Learning, in particular with Deep Learning, we have the big data regime where a lot of observations are available, albeit without any knowledge about the governing laws. Conversely, in the classical paradigm only the most essential data about the problem is known, that is the initial and the boundary conditions, however the equations describing the physics of the problem is completely known ([18]).

In a certain sense, the two paradigms represent the two extremes in terms of data and physics availability (fig 1.1.1). However, there is also an intermediate class of problems where the governing equations is partially known as well as there are some measurements of the system. ([18])
Thus, one of the main challenges in this scenario is to embed mathematical modeling in ML algorithms in order to obtain a more robust, interpretable and better performing model. As in [18], this process is also known as Physics-informed Machine Learning.

[18] mentions three biases that can guide a learning algorithm that favors solutions that are physically consistent (fig. 1.1.2). The first

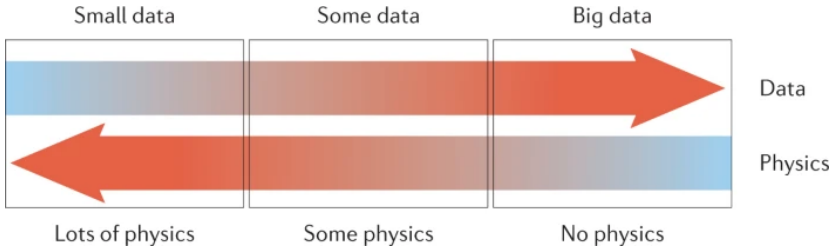

Figure 1.1.1: The spectrum of scenarios of data and physics availability. This figure is directly taken from [18]

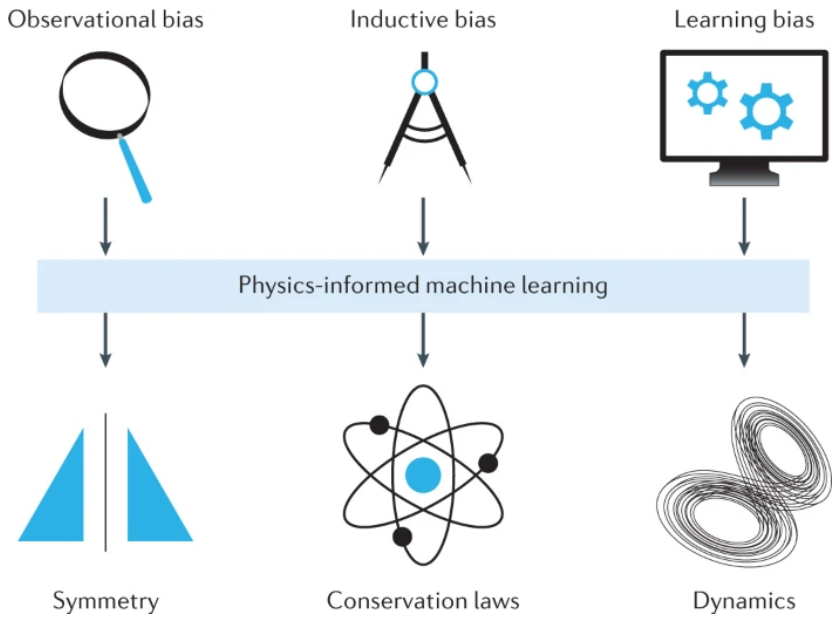

Figure 1.1.2: Three pathways to achieve a phyiscs-informed learning algorithm. This figure is directly taken from [18]

one are the observational biases, which is realized by directly introducing physically-consistent data; a way to do this is through the so-called process of data augmentation. Then there are inductive biases, which consist of modifying a ML architecture in a way that the physical constraints are followed. Finally, learning biases can be introduced by intervening the learning process of an algorithm: for instance, they can be introduced with an appropriate choice of the loss function.

Some instances of physics-informed learning algorithms are Physics-informed Neural Networks ([14]), Neural Ordinary Differential Equations ([11]) and Universal Differential Equations ([16]).

1.1.1 Universal Differential Equations

THEORY OF UDEs In [16], the *Universal Differential Equations* has been proposed as a hybrid model that is able to incorporate scientific knowledge (under the form of mechanistic models) into learning models.

To quote the same paper where the hybrid model is proposed, the definition of UDEs is as follows: *"Universal Differential Equations are differential equation models where part of the differential equation contains an embedded UA (Universal Approximator)"*.

In other words, we are incorporating learning models into the mathematical models themselves. In this work, we will focus on the motivating case of Ordinary Universal Differential Equations (UODEs), which is defined as follows:

$$x'(t) = f(x, t, U_\theta(x, t)) \tag{1.1}$$

In equation 1.1, f denotes a mathematical model, where some unknown dynamics are represented with the universal approximator U_θ. In this work, we will always take the approximal universal as a neural network.

Given a training set of points $\mathcal{D} \subseteq \mathbb{R}^+ \times \mathbb{R}^n$ represented as a time series, we can train an UODE in two main ways: the simplest way being to use differentiable numerical solvers, such as the Euler's method, and directly back propagate the loss to the universal approximator; otherwise, in [16] the author proposes a differentiable programming framework in order to handle more type of UDEs, such as Universal Stochastic Differential Equations, Universal Partial Differential Equations or Universal Delay Differential Equations.

The author of [16] argues that UDEs are robust and interpretable models that can be applied to problems related to classical modelling, such as identifying equations from a small dataset and accurately extrapolate the dynamics, or alternatively provide a surrogate for a faster, more accurate and consistent simulation of the dynamics.

CASE STUDY WITH PREDATOR-PREY DYNAMICS As such, in [16] the author proposes the first case study of UODEs, which entails as follows. Consider the following system of ODEs, also known as Lotka-Volterra equations:

$$\begin{aligned} \dot{x} &= \alpha x - \beta xy \\ \dot{y} &= \gamma xy - \delta y \end{aligned} \tag{1.2}$$

Where $\alpha, \beta, \gamma, \delta$ are positive factors.
Assuming that we only know the terms α, δ we can formulate the following UODE which replaces the interaction term xy with two neural networks:

$$\begin{aligned} \dot{x} &= \alpha x - U_1(x, y) \\ \dot{y} &= U_2(x, y) - \delta y \end{aligned} \tag{1.3}$$

Then as the UODE is trained, the author performed sparse symbolic regression with the SINDy algorithm ([9]) in order to recover the unknown interaction equations by using the learnt $U_{1,2}$ terms to obtain more training data and to estimate the derivative.

According to the supplementary material of [16], the following configuration has been used: 31 training data points have been sampled with constant step size $\Delta t = 0.1$, from the simulation of equation 1.2 with $\alpha = 1.3, \beta = 0.9, \gamma = 0.5, \delta = 1.8$ and $(x_0, y_0) = (0.44249296, 4.6280594)$. The UAs U_1, U_2 is an ANN consisting of one input layer, three hidden layers with 32 neurons each with the swish activation function (i.e. SiLU) and an output layer.
The UDE was initially trained for 1000 iterations with ADAM using the learning rate of 0.03, and then for 10000 iterations with gradient descent using the learning rate of 0.0004. The loss chosen for back propagation was the MSE L2 loss defined as the average of the square distances between the output and the data points in the L2 norm:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1,\dots,N} \|\hat{\mathbf{x}}(t_i) - \mathbf{x}_i\|_2^2 \tag{1.4}$$

Where $\hat{\mathbf{x}}$ is the solution of the initial value problem 1.3.

Moreover, the author trained a Neural Ordinary Differential Equation on the same dataset, with the same goal.

As a result, the author of [16] reported that UODEs was successful in recovering the original equations and extrapolating the dynamics, while the NDE-based approach failed to recover the correct coefficients and recovered the wrong terms of the dynamics.

Similar experiments have been proposed in [16], which leverage the use of Stochastic Differential Equations and other methodologies; as they are out of scope in this work, they are omitted.

1.2 Symbolic Regression with SINDy

One important problem that lies in the intersection between data-driven methods and mathematical modeling is system identification, which consists of identifying the underlying model and extrapolating its dynamics given some measurements of the system.

In [9], the authors present this problem under the assumption that only a few relevant terms compose the dynamics as the sparse identification of nonlinear dynamics (SINDy). More specifically, the paper considers the dynamical systems in the form of differential equations:

$$\dot{\underline{x}}(t) = F(\underline{x}(t)) \tag{1.5}$$

Where $\underline{x} \in \mathbb{R}^n$ represents the state of the system and F the underlying dynamics.

The goal of SINDy is to identify F as accurately as possible, given a time series of the states $\underline{x}(t)$ and its derivative $\dot{\underline{x}}(t)$.

To do this, we denote with $\mathbf{X}$ and $\dot{\mathbf{X}}$ as the matrix of observations, where each row represents an observation of the system under a certain timestamp. For example, $\mathbf{X}$ has the following structure:

$$\mathbf{X} = \begin{pmatrix} \underline{x}^T(t_1) \\ \vdots \\ \underline{x}^T(t_n) \end{pmatrix} \tag{1.6}$$

Obviously the structure is analogous for $\dot{\mathbf{X}}$.

Then we define the candidate of functions that will identify F as the matrix $\Theta(\mathbf{X})$, where each column represents the candidate term calculated on the dataset $\mathbf{X}$. For example, Θ can contain polynomial or trigonometric functions as a basis.

To determine the coefficients related to each term, we define Ξ as the vector of coefficients such that the model can be reconstructed as follows:

$$\dot{\mathbf{X}} \approx \Theta(\mathbf{X})\Xi \tag{1.7}$$

The matrix Ξ and its relative coefficient columns ξ_k are found by solving the following optimization, as formulated in [20]:

$$\min_{\Xi} \|\dot{\mathbf{X}} - \Theta(\mathbf{X})\Xi\|_2^2 + R(\Xi) \tag{1.8}$$

Where $R(\Xi)$ is the regularization term to enforce sparsity in the fit.

To summarize, the essential ingredients for a SINDy regression are as follows:

- A sample of observations $\mathbf{X}$
- The derivatives of each observation $\dot{\mathbf{X}}$; however, in most cases this term is unknown and the derivative is usually estimated with numerical derivation or other techniques
- A set of candidates defined in Θ
- An algorithm capable of solving sparse regression, such as the well-known LASSO regression ([4]) or the SR3 algorithm ([15])

1.3 Some Mathematical Models

As a famous mathematician once said, a mathematician is a maker of patterns[1]; that is, one of the main applications in mathematics is identifying patterns in everyday life and every field. One of the ways to concretize these patterns is with classical mathematical modeling, which translates real-world phenomena into equations. In this section, we will have a brief overview of differential equations that model the most important systems in biology, neurosciences and electric engineering.

[1] G. H. Hardy, *A Mathematician's Apology* (1940):

> A mathematician, like a painter or poet, is a maker of patterns. If his patterns are more permanent than theirs, it is because they are made with ideas

1.3.1 Population Biology

The study of population growths is the most historical instance of applications of mathematical modeling, starting in 1202 with a mathematical model for a growing rabbit population written in an arithmetic book by Leonardo of Pisa ([5]).

Here we will have a very brief overview on the models describing population growths, which will be subject to application in this thesis.

VERHULST LOGISTIC MODEL In 1978, Malthus described one of the simplest models of population growth ([5]):

$$\dot{N} = bN - dN = \rho N \tag{1.9}$$

Where b, d, ρ are respectively the birth, death and reproduction rate of the population. For $N(0) = N_0$, the model described in equation 1.9 yields the following solution:

$$N(t) = N_0 e^{\rho t} \tag{1.10}$$

This suggests an exponential growth of the population, which is fairly unrealistic. In fact, the crucial shortcoming of the model is the fact that it assumes that the resources are infinite, even on the long term.

So, in 1838 Verhulst proposed the model that limits itself when it becomes too large due to the lack of resources with the following model ([5]):

$$\dot{N} = rN(1 - N/K) \tag{1.11}$$

Where $r > 0$ is the reproduction rate of the population and $K > 0$ is the carrying capacity, that is the limit of the growth for the population. Equation 1.11 also yields an open-form solution:

$$N(t) = \frac{N_0 K e^{rt}}{K + N_0(e^{rt} - 1)} \tag{1.12}$$

In 1845, Verhulst named the equation 1.12 as the logistic equation ([5]).

One of the possible further generalizations of the equation 1.11 consists of adding further terms in the second factor of higher degree. In particular, we can replace that term with a polynomial with positive coefficients:

$$\dot{N} = rN(1 - \rho_1 N - \rho_2 N^2 - \ldots - \rho_k N^k) \tag{1.13}$$

Even more generally, we can rewrite equation 1.13 as the following DE with a polynomial on the right hand side:

$$\dot{N} = q(N) \tag{1.14}$$

With $q \in \mathbb{R}[N]$ a polynomial such that $q_0 = 0, q_1 = r$ and $\forall i \geq 2, q_i < 0$.

SPRUCE BUDWORM OUTBREAK MODEL [2] proposes a model for the growth of a population that interacts with an external environment that preys upon the population, according to its density. It is an extension of the logistic model (eq. 1.11) that is subtracted with a functional form $\varphi(x)$ that saturates for $x \to +\infty$:

$$\dot{x} = rx(1 - x/K) - \beta \frac{x^2}{\alpha^2 + x^2} \tag{1.15}$$

It can be extensively shown that the model presents a hysteresis bifurcation according to the parameter β (fig. 1.3.3); that is, there exists two critical values β_1, β_2 such that for $\beta < \beta_1$ and $\beta > \beta_2$, the model presents one stable fixed point; however for $\beta \in (\beta_1, \beta_2)$ the model presents two locally asymptotically stable fixed points and one unstable equilibrium.

LOTKA-VOLTERRA MODEL Another important class of biological models are the predator-prey dynamics, where we have one population of "preys" x and one population of "predators" y that obey the following equation:

$$\begin{aligned} \dot{x} &= \rho_x(x, y)x \\ \dot{y} &= \rho_y(x, y)y \end{aligned} \tag{1.16}$$

Where $\rho_{x,y}$ are the respective growth rates and η is the conversion rate.

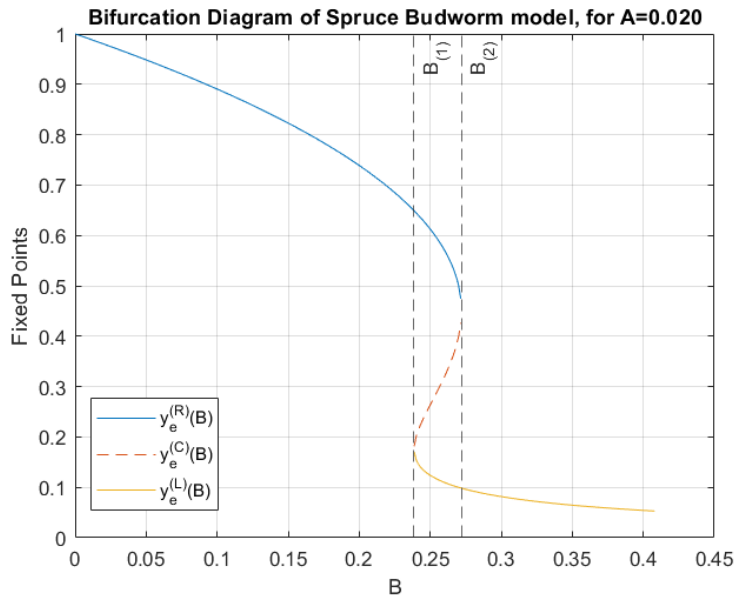

Figure 1.3.3: Bifurcation diagram of the Budworm model (eq. 1.15) for $\alpha = 0.02$

In 1926, Volterra proposed the simplest realization of equation 1.16 making a linear choice for the growth rates ([5]):

$$\begin{aligned} \dot{x} &= x(\alpha - \beta y) \\ \dot{y} &= y(\gamma x - \delta) \end{aligned} \tag{1.17}$$

The most notable characteristic about the model described in equation 1.17 is that the system presents periodic solutions, orbiting around the fixed point of the equation (fig. 1.3.4).

Further generalizations of the Lotka-Volterra equations (eq. 1.17) can account for other sort of dynamics, such as in [22] with psychological fear effect and resource limitations.

1.3.2 Neurons

One major field of interest in mathematical modeling is neurosciences, in particular modeling and studying the dynamics of neurons and network of neurons. Recently there have been major advancements in this field, such as the full-scale simulation of the adult Drosophila fly brain ([21]), motivating this case study even further.

In [19], two kinds of neuron models have been proposed as case

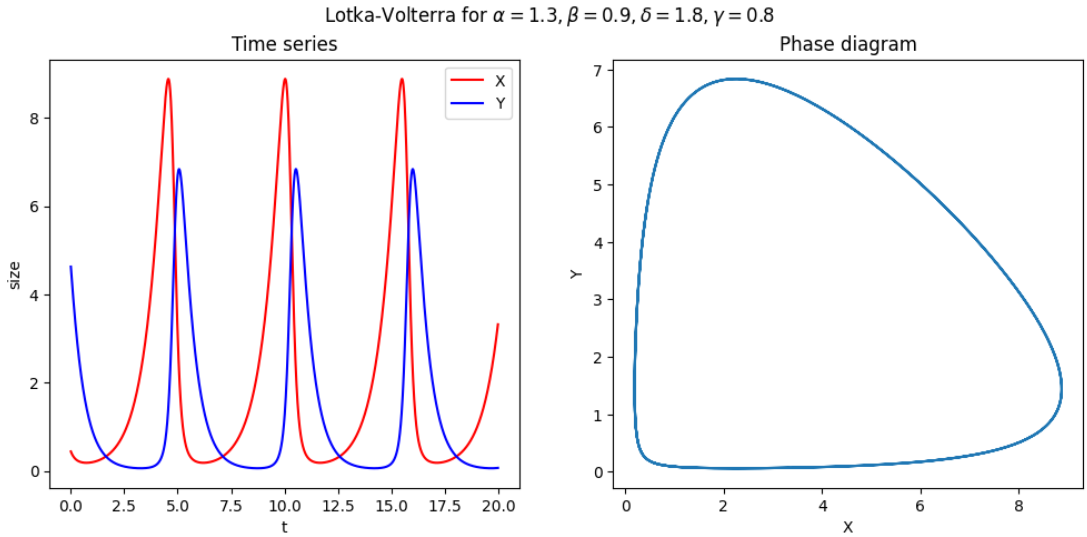

Figure 1.3.4: Periodic Solution of Lotka-Volterra model (eq. 1.17)

studies: rate models and network of spiking neurons. In this work, we will mainly see the first class of neuronal models.

RATE MODELS Rate models consist of modeling neurons as a population with a continuous variable that obeys an ODE ([19]), with the variable representing the mean activity of the neurons. The simplest form of a rate model can be described with the following ODE:

$$\tau \dot{r} = -r + \Phi(I_{\text{ext}} + Jr) \tag{1.18}$$

Where in the equation 1.18, τ refers to a time constant of the dynamics, Φ is a non-linear transfer function which transforms impulse to firing rate, I_{ext} is the external impulse of the population and J is the strength of the intra-population connections.

We say that the population is excitatory for $J > 0$, inhibitory otherwise. The Φ function can take different forms, with the most popular case being the threshold-linear function (also known as ReLU):

$$\Phi(I) := \max\{0, I - T\} \tag{1.19}$$

Where $T > 0$ of equation 1.19 is a threshold value. In other cases, it can be also defined as a sigmoidal function, as follows:

$$\Phi(I) := \frac{r_{\max}}{1 + e^{-\beta(I-T)}} \tag{1.20}$$

Notice how the equation 1.20 converges to the following function for $\beta \to +\infty$:

$$\lim_{\beta \to +\infty} \frac{r_{\max}}{1 + e^{-\beta(I-T)}} = \begin{cases} r_{\max}, & I > T \\ 0, & I < T \end{cases} \tag{1.21}$$

Given more populations of neurons $r_1, \ldots, r_n$, we can extend the firing rate of singular populations as described in equation 1.18 into a model describing the network of coupled neurons.

In [8] the following model has been proposed:

$$\forall i = 1, \ldots, n; \tau \dot{r}_i = -r_i + \Phi\left(\sum_{j=1,\ldots,n} w_{ij} r_j + I_{\text{ext},i}\right) \tag{1.22}$$

Where w_{ij} represents the bond between the population i and j.

It is interesting to observe how in [8] the model described in 1.22 was derived from a generalization of the logistic Lotka-Volterra equation as an analogy of neural populations.

As proposed in [19], an alternative way to extend the single-population firing rate model (eq. 1.18) is to introduce a continuous variable x that represents the "location" of the population and rewrite the model as the following integro-differential equation (IDE):

$$\tau \frac{\mathrm{d}r}{\mathrm{d}t}(x,t) = -r(x,t) + \Phi\left(I_{\text{ext}} + \int_\Omega J(x,y) r(y,t)\, \mathrm{d}y\right) \tag{1.23}$$

However, we will prefer the "discrete" approach (i.e. the model described in equation 1.22) in the thesis as it provides a system of ODEs rather than an IDE.

INTRINSIC AND STIMULUS-DRIVEN DYNAMICS In order to study network dynamics, there are two approaches: intrinsic network dynamics consists of assuming constant and uniform external impulse; whereas the stimulus-driven approach observes the behavior of the network when the external impulse is represented as a time-valued function ([19]).

WINNER TAKE ALL MECHANISM One particular case study of the intrinsic network dynamics is the so-called winner-take-all mech-

anism as described in [19]. Consider the following realization of equation 1.22:

$$\begin{aligned} \tau \dot{r}_1 &= - r_1 + \Phi(I_0 - J r_2) \\ \tau \dot{r}_2 &= - r_2 + \Phi(I_0 - J r_1) \end{aligned} \tag{1.24}$$

Where $J > 0$ describes the recurrent inhibition.

It can be shown that the system of DEs 1.24 presents the homogeneous steady state, where $r_1 = r_2 \equiv r_s$ such that $r_s = \Phi(I_0 - r_s)$. However, for a sufficiently large value of J it can be also shown that such steady state is unstable and instead we reach an equilibrium where the activity of either sub-network is enhanced and the other sub-network is suppressed.

TODO: abbozzare una dim (??, non necessaria forse)

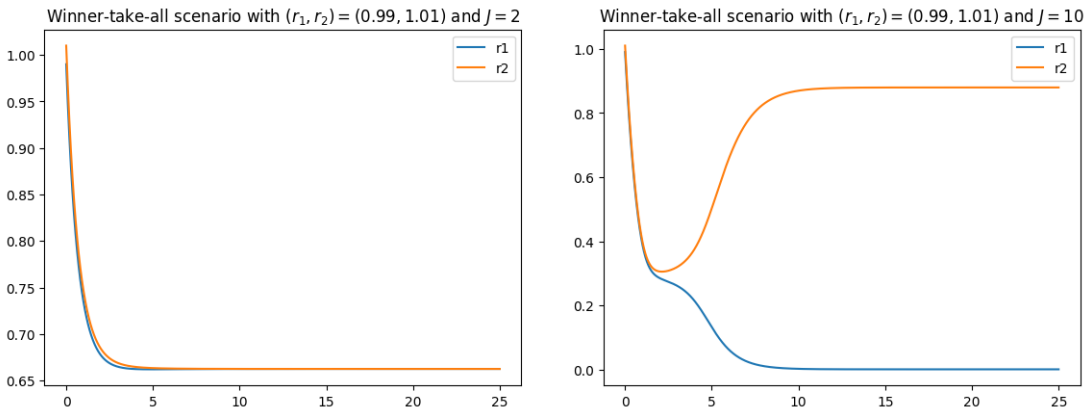

Figure 1.3.5: Two steady states of equation 1.24

1.3.3 Chua's Circuit

DA FARE QUANDO INIZIO CON QUESTA PARTE)

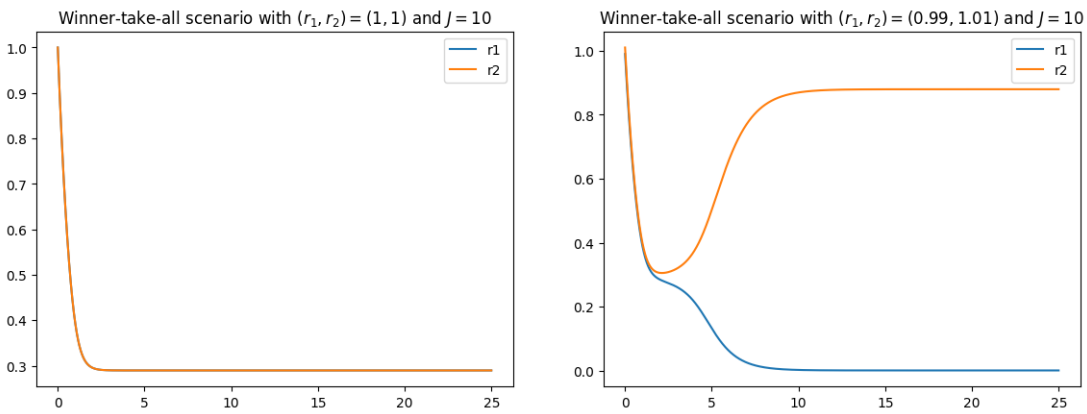

Figure 1.3.6: Instability of the homogeneous steady state under large J of equation 1.24

Chapter 2

Methodology and Reproduction of the Main Results

In this chapter, we will be reviewing the methodologies to implement the Universal Differential Equations as described in [16].

The chapter will begin with a simple implementation using Euler's method as an IVP solver in order to learn the interaction of the budworm model (eq. 1.15); then, it will leverage symbolic regression (subsec. 1.2) to learn the polynomial term of the generalized logistic equation (eq. 1.14) enhanced through the use of UDEs.

In the end, this chapter culminates with the reproduction of the Lotka-Volterra experiment of [16] (para. 1.1.1), with some slight modifications.

2.1 The Spruce Budworm Model

Recall the bud worm outbreak model as in equation 1.15:

$$\dot{x} = r_0 x(1 - x/K) - \underbrace{\frac{\beta x^2}{\alpha + x^2}}_{\varphi(x)} \tag{2.1}$$

Suppose that a scientist sampled a time series of this model, however does not know the interaction term $\varphi(x)$. Then they can use the Universal Differential Equations to approximate the term $\varphi(x)$ with a neural network $U(x;\theta)$. Then, we obtain the following UDE:

$$\dot{x} = r_0 x(1 - x/K) - U(x;\theta) \tag{2.2}$$

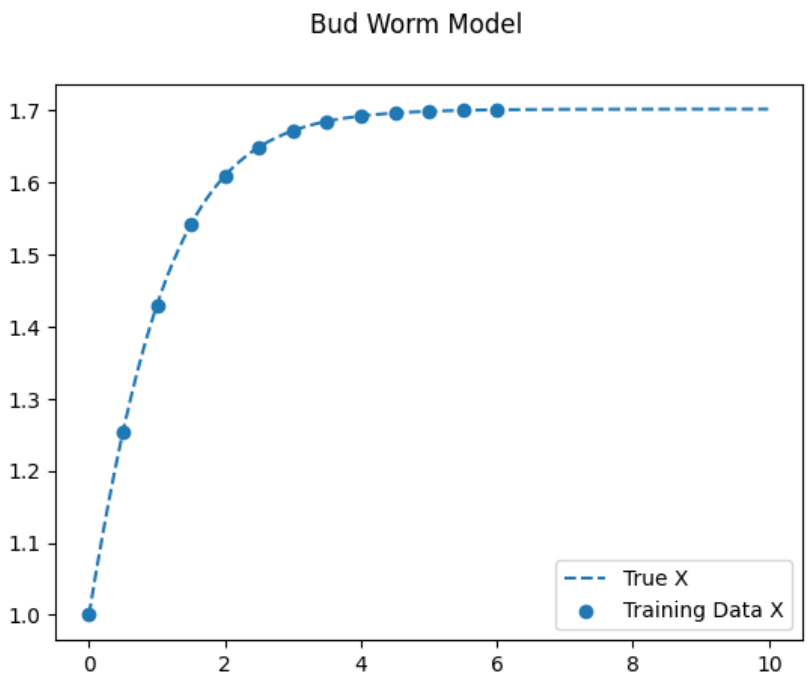

Figure 2.1.1: Budworm Model Training Data

In the simulation of the model (fig. 2.1.1), the following parameters have been used:

- The initial condition is set to $x_0 = 1$
- The parameters of the original model are set to $r_0 = 2, K = 3, \alpha = 3, \beta = 3$
- The training data consists of a trajectory of 13 data points with a constant step size $\Delta = 0.5$

Then, to train the Universal Differential Equation the following methods were employed:

- The neural network is a fully-connected neural network consisting of an input layer, a hidden layer with 32 neurons with the SiLU activation function and an output layer.
- The solver used to solve the IVP 2.2 will be the Euler's method, defined as $x(t_{k+1}) = x(t_k) + hF(U, x, t)$. In particular, the implementation of the Euler algorithm will be compatible with the Pytorch methods for automatic differentiation.
- The model has been trained with the ADAM optimizer for 3000 epochs with a learning rate of 0.01

In the end, we obtained a model that fits the training data with a final loss of 6.7945×10^{-7}. By observing the graphs of the learned

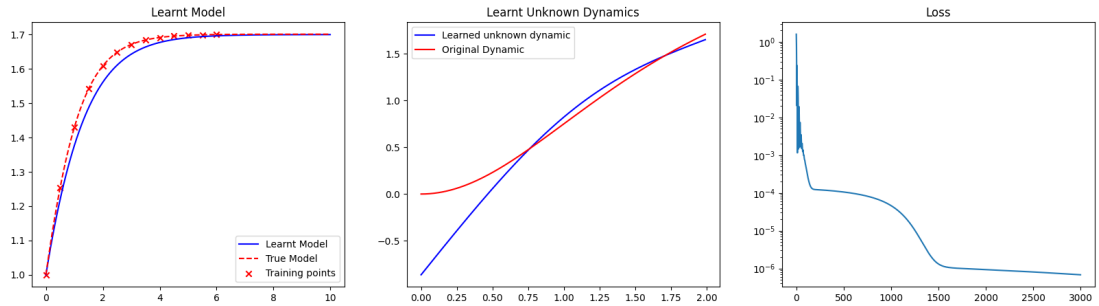

Figure 2.1.2: Budworm UDE Results

trajectories and unknown dynamics (fig. 2.1.2), we can indeed claim that UDEs hold the potentialities to learn and extrapolate dynamics from small datasets and partially unknown models.

So far, this was an outline of the methodology used to study models with UDEs: to improve this method even further and use it in more advanced contexts, we will consider using other differentiable IVP solvers (such as the ones provided in [10]), tuning the neural network with an appropriate architecture or considering symbolic regression for dynamics extrapolation.

2.2 Modeling Generalised Logistic Growths

Now, as a slightly more advanced case study let us consider the generalized logistic growth equation, which we defined as previously with equation 1.13:

$$\dot{x} = r_0 x(1 - \rho_1 x - \rho_2 x^2 - \ldots - r_k x^k) \tag{2.3}$$

We have performed (???) experiments on this model, with the aim of studying the capabilities of UDEs and SINDy of handling this scenario.

EXPERIMENT 1 (LEARNING POLYNOMIALS) In this experiment, the set-up of the experiment is as follows:

- We take $k = 4$, with parameters $r_0 = 1, \underline{\rho} = [0, 0.25, 0.1, 0.001, 1]$
- The training data consists of only three data points, with $t = 0, 1, 3$. The initial state is set to $x(0) = 1$
- We assume the polynomial term to be unknown, so our UDE will be defined as $\dot{x} = r_0 x(U(x; \theta))$

- To solve the UDE, the Dormand-Prince method of order 5 (DOPRI-5) has been employed
- The neural network is composed of an input layer, a hidden layer with 10 neurons, a hidden layer with 32 neurons, a hidden layer with 10 neurons and an output layer. Each hidden layer uses the hyperbolic tangent activation function (also known as tanh)

The objective of the experiment is to: (1) train an UDE that is able to reproduce the original dynamics, (2) given the trained UDE, infer the original polynomial with the SINDy method.

For the objective (2), we will train three SINDy models:

- The first model is trained directly on the training data, approximating the derivative with the finite-difference method (see [20]). This serves as a baseline model, and will be used to compare with the next models
- The second model consists of a UDE-enhanced approach, as the derivative is approximated with the learnt UDE. The training set remains the same
- The third and last model is the same as previous, however the UDE is also leveraged to obtain more training data for the sparse regression; in particular, we will have a more dense sampling with a constant time step $\Delta t = 0.01$, from $t = 0$ to $t = 3$.

We trained the UDE with the ADAM optimizer for 1000 epochs with a learning rate of 0.01, yielding a final loss of 6.9468×10^{-8}. By also observing the learned dynamics on a longer time horizon (fig. 2.2.3), we can conclude that the objective (1) has been satisfactorily realized.

Then, for objective number (2), we trained the three SINDy models, yielding their respective final objective values (as defined in equation 1.8), coefficients and distance from the original polynomial quantified as the L^2 distance, defined in equation 2.4.

$$d_{L^2([0,1])}(f, g) := \int_0^1 |f(x) - g(x)|^2 \, \mathrm{d}x \tag{2.4}$$

As we can observe in the figure 2.2.4 and in the tables 2.2.2, 2.2.1, we can clearly conclude that the SINDY-enhanced approach managed

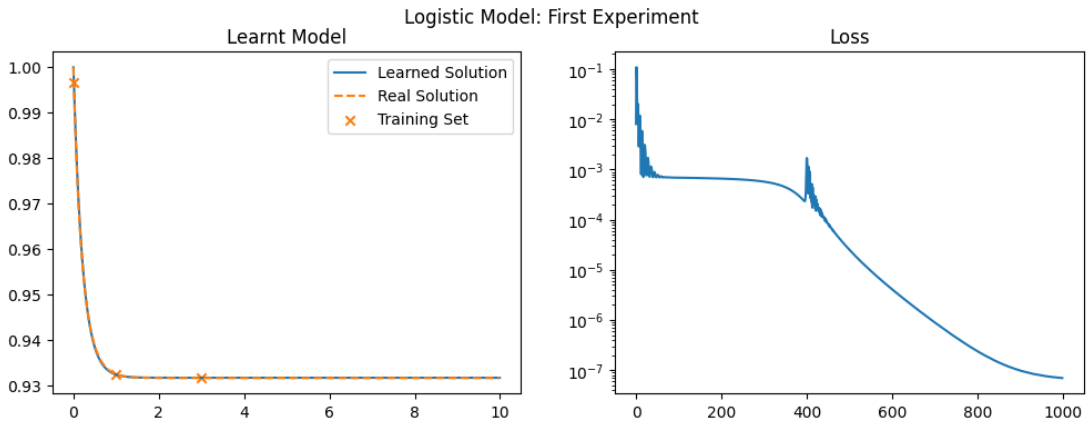

Figure 2.2.3: Generalized Logistic First Experiment Results

to learn very accurately the target polynomial function as it reduced the baseline L^2 error from 3.593×10^{-2} to 8.052×10^{-5}. However, the approach in leveraging UDEs to generate more data points for the sparse regression turned out to be worse than the previous approach; nonetheless, it still managed to improve the baseline result, reducing the baseline error to 2.135×10^{-3}.

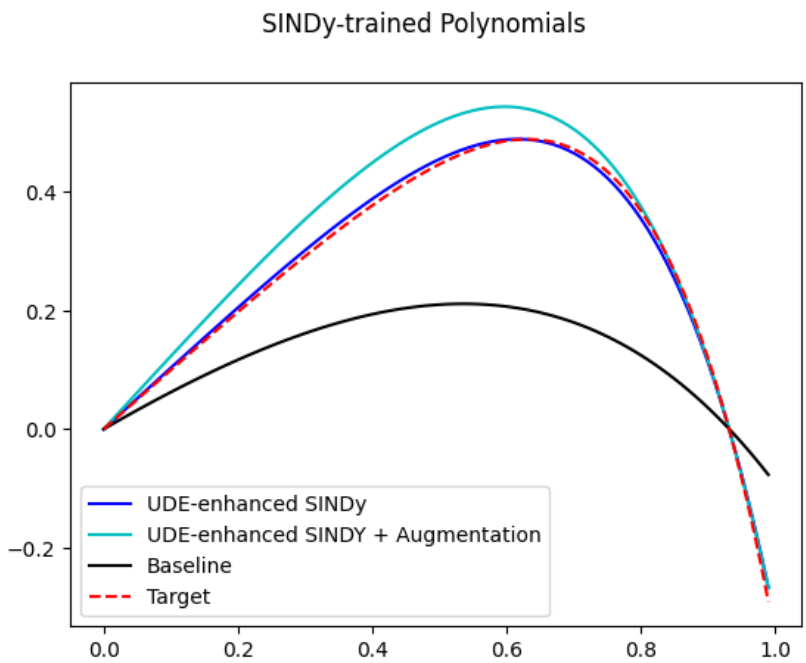

Figure 2.2.4: Generalized Logistic First Experiment Results (SINDy)

EXPERIMENT 2 (CHANGING HYPERPARAMETERS) We can observe that the polynomial defined in the previous experiment has two

Name	**Total Error**	L^2 **distance**
Original Polynomial	0×0^{128}	0×0^{128}
Baseline	3.3612×10^{-2}	3.593×10^{-2}
UDE-enhanced	3×10^{-2}	8.052×10^{-5}
UDE-enhanced + Data Augmentation	3.0157×10^{-2}	2.135×10^{-3}

Table 2.2.1: Model comparison metrics

Name	**Coefficients**
Original Polynomial	$[1,\ 0,\ -0.25,\ -0.1,\ -0.001,\ -1]$
Baseline	$[0.655,\ -0.295,\ -0.248,\ -0.203,\ 0,\ 0]$
UDE-enhanced	$[1.035,\ 0,\ -0.193,\ -0.295,\ -0.39,\ -0.478]$
UDE-enhanced + Data Augmentation	$[1.234,\ 0,\ -0.414,\ -0.489,\ -0.423,\ -0.230]$

Table 2.2.2: Recovered coefficients

roots: one at $x = 0$ and the other one at $x \approx 0.9315$. With a simple qualitative analysis of the graph of the dynamics (fig. 2.2.5) we can see that the equilibrium at $x = 0$ is unstable and the other one is globally stable. Moreover, in the previous experiment we set the initial state at $x_0 = 1$, which is fairly close to the global attractor. We considered the idea to study the same model for different initial states, with the goal to observe how the model performs for different initial conditions

Also we used a very small amount of training data, with only three points; an analogous consideration as above can be done for this case as well.

In this experiment, we will generalize the pipeline of the previous experiment in order to (1) observe how differently will the models perform for different initial conditions and (2) for different sizes of training data and for a fixed initial condition $x_0 = 1$

The set up of the experiment will entail as the following: for (1), we will take 50 equally-spaced values for $x_0 \in [0.01, 2]$. For each different initial point, the training data will consist of a trajectory of ten data points, ranging from $t = 0$ to $t = t_\infty + 4$, where t_∞ is the timestamp where can consider that the population size has reached its steady state. We define t_∞ as the first time instant such that the l^1 distance between $x(t)$ and the steady state is smaller than an

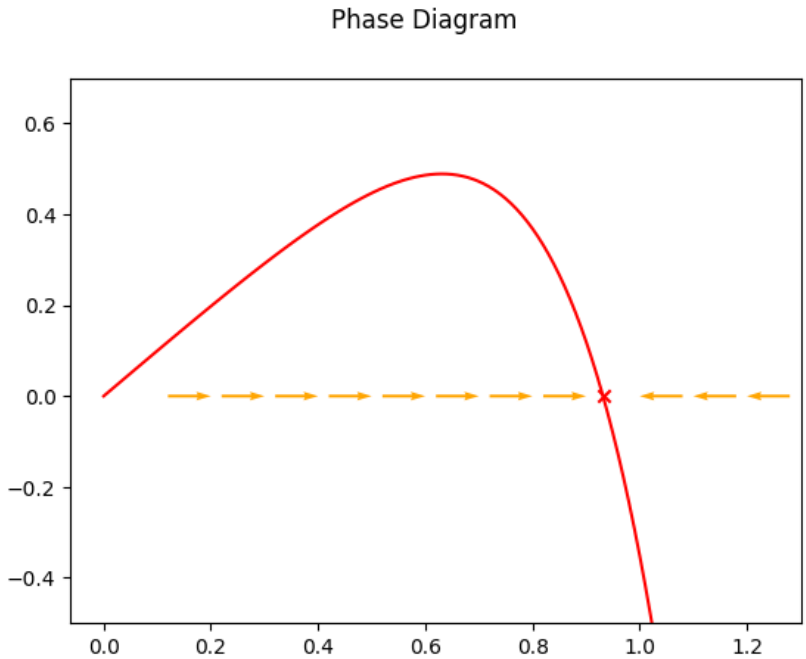

Figure 2.2.5: Phase diagram of the generalized logistic model

arbitrarily small ε:

$$t_\infty := \min_{t \geq 0} |x(t) - x_\infty| < \varepsilon \tag{2.5}$$

In our case, we set $\varepsilon = 1 \times 10^{-4}$. Figure 2.2.6 illustrates the generated time series with the above-described methodology. Then for (2) we set $x_0 = 1$ and considered $N \in [3, 29] \cap \mathbb{N}$ for $t \in [0, 3]$
The figures 2.2.7, 2.2.8 report the final losses and the L^2 errors of the trained SINDy models for different values of x_0 and n.

In the first case, we can see that the UDE-enhanced method works generally better than the baseline model for low x_0 values. However, for the values near the initial condition the UDE-enhanced method turned out to be slightly worse than the baseline method, which is opposite of what happened in the previous experiment. This could be due to the fact that we have sampled the training set differently, especially in terms of the training dataset. Then we can also observe that for higher x_0 values, the baseline method suffers a spike in its L^2 error, while the UDE-enhanced method remains stable with an L^2 error of around 10^{-2}.

For the second case, we can see that from around 10 data points

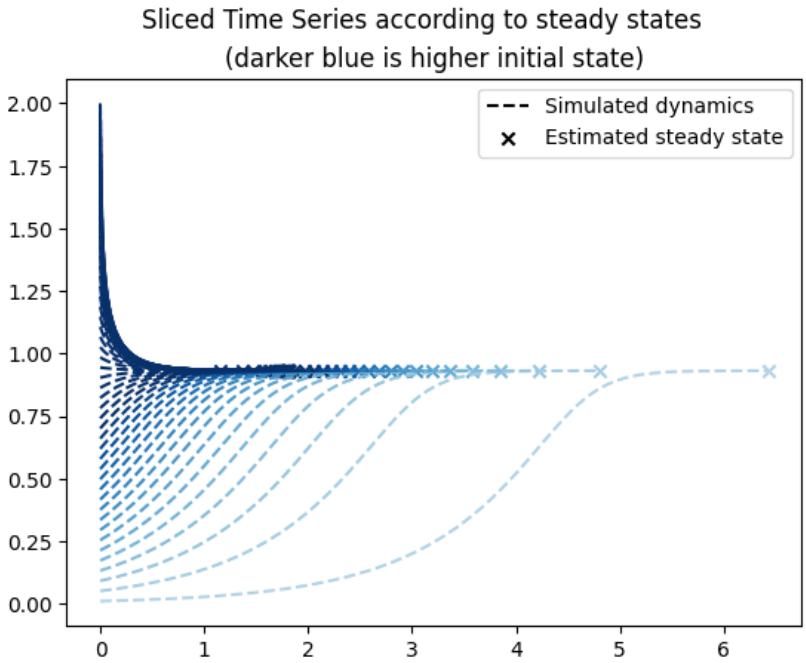

Figure 2.2.6: Generated trajectories for different initial points

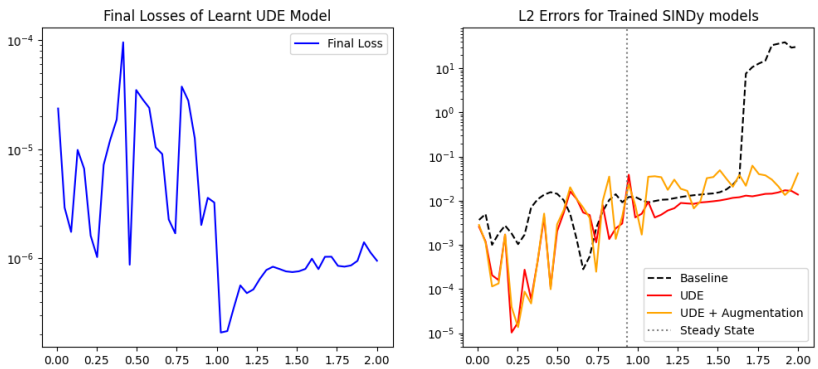

Figure 2.2.7: Results of second experiment, for $x_0 \in [0.01, 2]$

onward the UDE-enhanced method tends to destabilize, suggesting that the method works better for a moderate size of training set.

In summary, with this experiment we can conclude that the UDE-enhanced SINDy method can sometimes improve than the base model, although it is robust for a moderate amount of training samples and for different initial conditions of the model.
TODO: DA DISCUTERE

EXPERIMENT 3 (TRAINING UDES WITH MORE TRAJECTORIES) In the previous two experiments, we have trained the UDE model us-

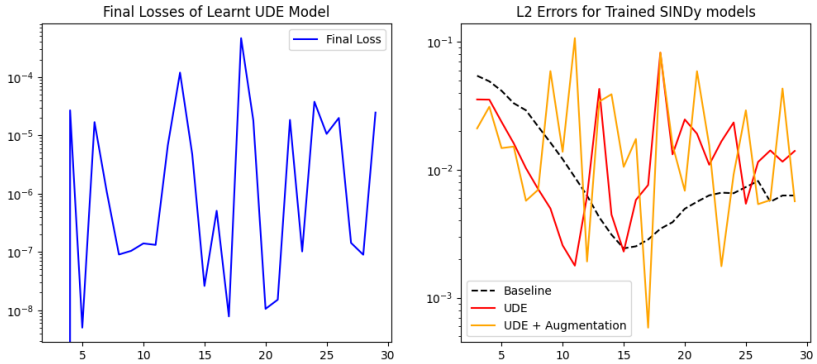

Figure 2.2.8: Results of second experiment, for $n = 3, \ldots, 29$

ing only one timeseries of the model: however, a scientist might have multiple trajectories of the same system. So an idea to create a more robust model, especially with respect to unseen initial conditions, is to train the models with multiple trajectories of the model.

In this experiment, the goal is to (1) see how the UDE-enhanced SINDy method changes between the two training methods and (2) compare the extrapolated dynamics of the UDEs by comparing their trajectories with an unseen initial condition.

In this experiment we will consider the same coefficients of the first experiment, however we will consider three trajectories of the model with initial conditions $x_0 = 0.5, 1, 1.5$ with five data points each. Then we have trained two UDEs models: one uses only one time series for training, the other one uses all of the three trajectories. The first model was trained for 5000 epochs with a learning rate of 1×10^{-3} and the second one was trained for 1000 epochs with the same learning rate. For both of the UDEs, the RK4 solver was used. The SINDy models are trained with the same training datasets,

In figure 2.2.9 we can see that the two models have very similar performances, with the UDE-enhanced model that has been trained on multiple trajectories yielding an L^2 error of 2.065×10^{-2}, while the one trained on only one trajectory yielded an error of 2.408×10^{-2}. We can consider this to be only a very slight improvement of the UDE-enhanced SINDy method for the goal (1).

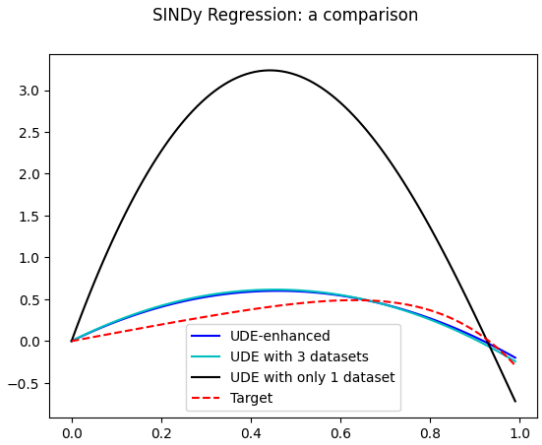

Figure 2.2.9: Third experiment: SINDy models

However, for objective (2), we see from figure 2.2.10 that the base UDE is not able to generalize the dynamics for unseen initial conditions as well as the other UDE trained with multiple trajectories, especially considering the unseen initial condition $x = 0.01$ for both trained UDEs. Therefore, we can still infer that this form of training to be an improvement of UDEs in terms of dynamic extrapolation and robustness: the only caveat is the increased computational expenses and requirements for more data.
TODO: DA DISCUTERE

2.3 Lotka-Volterra Predator-Prey Equations

In [16], the Lotka-Volterra equations have been considered as a case study for UDEs and SINDy symbolic regression. Let us recall the equations (eq. 1.17 and the parameters used in the supplements of [16]:

$$\begin{aligned} \dot{x} &= \alpha x - \beta xy \\ \dot{y} &= -\delta y + \gamma xy \\ (x_0, y_0) &= (0.44249296, 4.6280594) \\ \alpha &= 1.3, \beta = 0.9, \gamma = 0.8, \delta = 1.8 \end{aligned} \tag{2.6}$$

With the model described in equation 2.6, we will perform three experiments:

- The first experiment is aimed at reproducing [16], with a very similar methodology

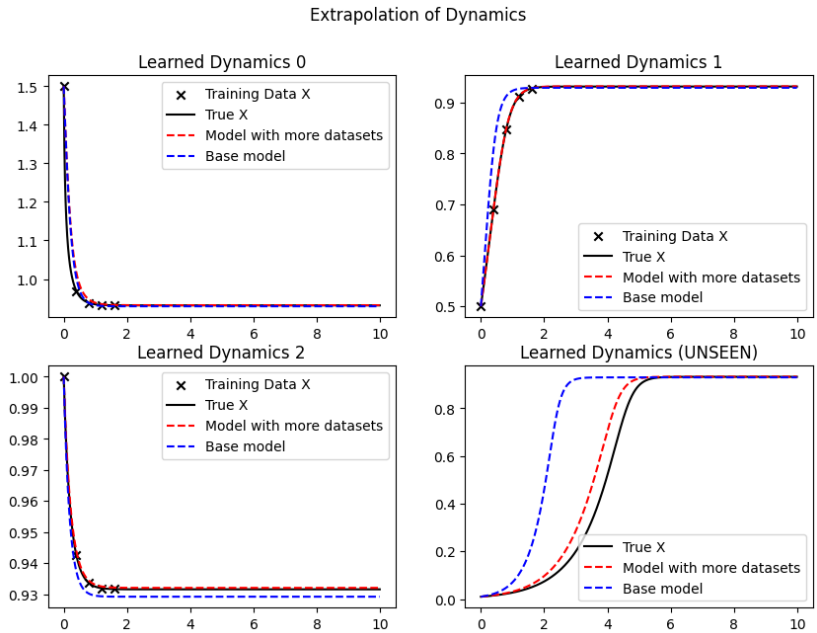

Figure 2.2.10: Comparison of third experiment's models

- The second experiment is aimed at reproducing [16] again, however with some substantial differences from the original methology
- The third experiment changes the parameters of the equation 2.6, in particular changing the constants associated to the interaction term xy.

EXPERIMENT 1 (REPRODUCTION OF [16]) In this experiment, we will follow the a similar methodology as covered in [16]. One slight difference is in the UDE's Neural Network architecture, where in addition to the proposed layers, we have added an extra linear layer on the output layer which compresses the output layer to a single variable and from which we obtain a 2-dimensional output with another linear layer. This is done to encode the idea of learning the parameters β, γ implicitly.

Another difference is that we trained the UDE for 1000 epochs with the ADAM optimizer with a learning rate of 8×10^{-3}.

For the rest, let us recall the same methodologies proposed in [16]:

- In addition to the UDE, a Neural Ordinary Differential Equation is trained on the dataset

- The loss is the L^2 mean-squared error metric, that is the one calculated according to equation 2.7
- Three SINDy models will be trained: one is the base SINDy model, the other two are UDE and NDE-enhanced SINDy models where the derivative are calculated with the learnt models
- The SINDy models will be optimized with the Sequentially Thresholded Least Squares (STLSQ) algorithm
- The obtained coefficients from the UDE-enhanced SINDy model will be improved with a post-structure identification parameter estimation method. In our case, we will optimize the loss function defined as the sum of squared errors in the x and y components

$$\mathcal{L} := \frac{1}{N} \sum_{i=1,\dots,N} \|\hat{\mathbf{x}}(t_i) - \mathbf{x}(t_i)\|_2^2 \tag{2.7}$$

The trained UDE model yielded a final loss of 9.759×10^{-4}, while the NDE yielded a final loss of 3.363×10^{-5}. The recovered coefficients with SINDy are reported in table 2.3.3

Name	**Recovered $\dot{x}$**	**Recovered $\dot{y}$**
Original LV equations	$1.3x - 0.9xy$	$-1.8y + 0.8xy$
Original SINDy	$1.248x + 0.124x^2$ $- 0.885xy - 0.066x^3$	$-1.8y + 0.787xy$
UDE-enhanced SINDy	$1.064x - 0.791xy$	$-1.949y + 1.350xy$
UDE + Parameter Estimation	$1.30000148x - 0.90000097xy$	$-1.8000203y + 0.80007929xy$
NDE-enhanced SINDy	$2.625x - 0.306y - 3.793x^2$ $+ 2.656x^3 - 0.429x^5$	$-1.71y + 0.279x^2 - 0.491xy$

Table 2.3.3: Recovered coefficients

Despite a lower performance in terms of the L2 MSE loss, the UDE-enhanced method managed to correctly recover the terms with a slight discrepancy in the recovered coefficients; however, with an estimation parameter method using the cofficients as an initial guess, we obtained new coefficients that are almost perfectly identical to the original equation. As we can see in figures 2.3.11 and 2.3.12, UDEs are able to extrapolate the dynamics of Lotka-Volterra equations from a short training set.

Although the original SINDy method was fairly accurate in obtaining the coefficients of the system, it also incorrectly introduced some

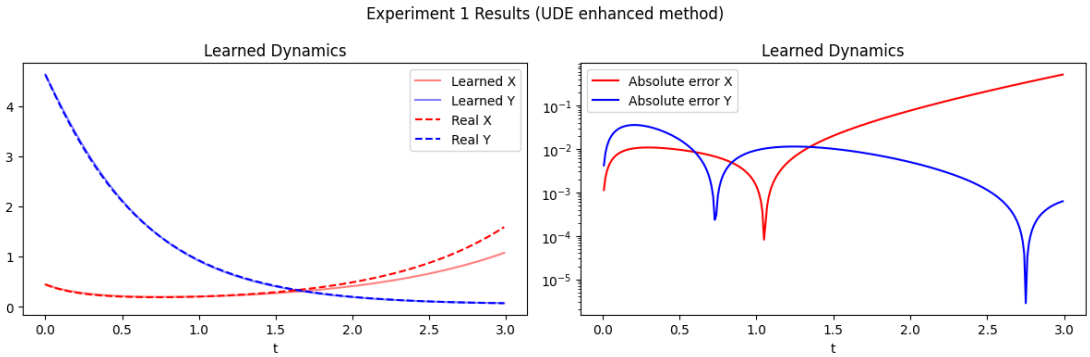

Figure 2.3.11: Simulation of the UDE-enhanced SINDy recovered model

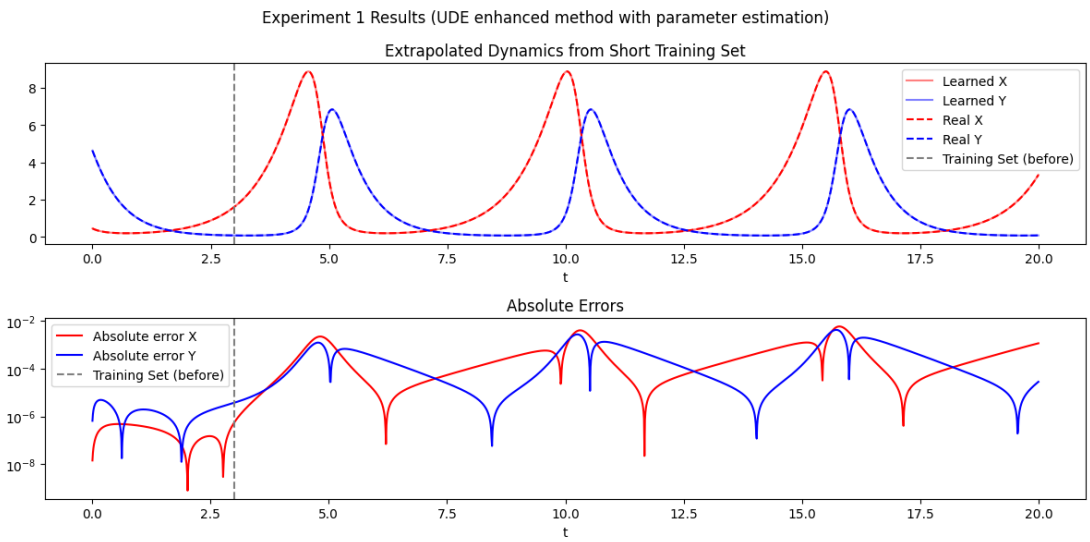

Figure 2.3.12: Simulation of the UDE-enhanced SINDy recovered model, with post-structure identification parameter estimation

other factors in the equation for $\dot{x}$, in particular the quadratic and cubic term. This is a rather critical problem, as with such result the scientist can wrongly infer the dynamics of the extrapolated system. To give an idea of this grave error, let us write the full equations for the base SINDy-recovered dynamics:

$$\dot{x} = 1.248x + 0.124x^2 - 0.885xy - 0.066x^3 \tag{2.8}$$

$$\dot{y} = -1.8y + 0.787xy \tag{2.9}$$

Assume that the two populations live in a predator-free scenario, that is we set $y_0 = 0$. Then with some simplifications of equation 2.8 we obtain the following growth model for the prey population:

$$\dot{x} = x(1.248 + 0.124x - 0.066x^2) = x(b(x) - m(x)) \tag{2.10}$$

Where $b(x)$, $m(x)$ indicate the birth and death rate of the population. By plotting the birth and death rates (fig. 2.3.13) we can observe a single intersection at around $x \approx 5.388$ which indicates a weak Allee effect of the population. Unfortunately, this is completely wrong as the original model did not intend such effect, indicating instead exponential growth for both of the populations x, y.

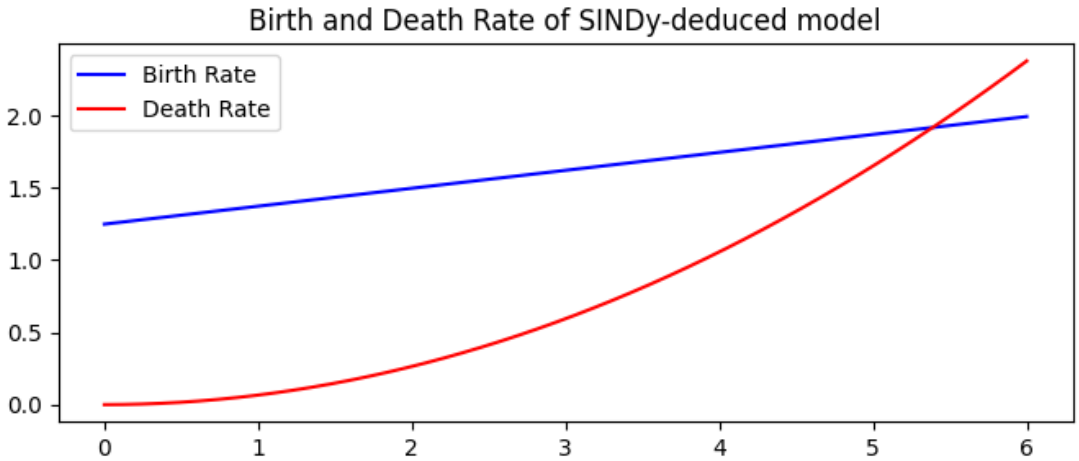

Figure 2.3.13: Birth and Death Rates of the wrongly inferred model

On the other hand, the NDE-enhanced method incorrectly returned more coefficients, especially in the first equation where also the interaction term is missing; this kind of mistake leads to an analogous observation as done previously. This confirms that the knowledge-enhanced method is more robust and better than the full neural network approach (i.e. NDE), as claimed in [16].

With this experiment, we managed to successfully reproduce the Lotka-Volterra case study of [16]. However, a better choice in NN architecture can be made to leverage our knowledge of the dynamics even further which will be done in the following experiment.

EXPERIMENT 2 (ALTERNATIVE UDE ARCHITECTURE) This experiment's set up is essentially the same as the previous one, the substantial difference is in the architecture of the UDE. In the previous

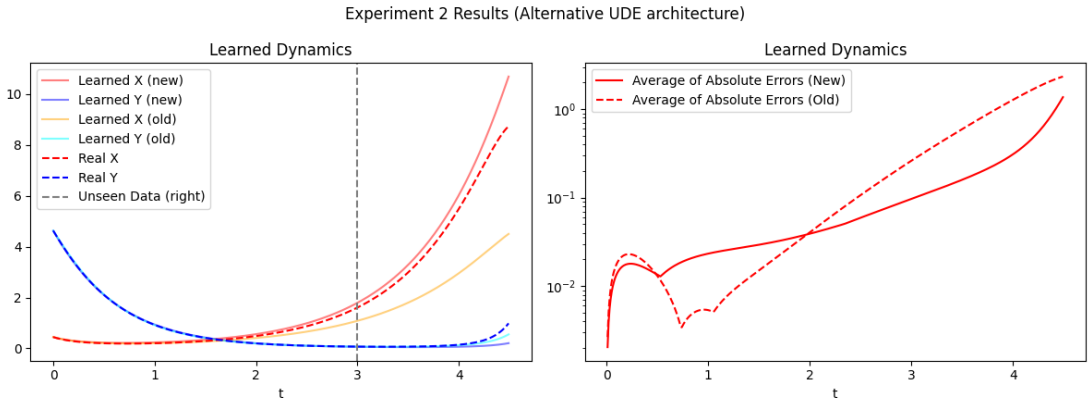

Figure 2.3.14: Comparison of the alternative UDE architecture with the previously proposed one

experiment, the UDE was defined with the following equations:

$$\begin{aligned} \dot{x} &= \alpha x - U^{(1)}(x, y; \theta) \\ \dot{y} &= -\beta y + U^{(2)}(x, y; \theta) \end{aligned} \tag{2.11}$$

In other words, we had a neural network with two outputs. However, this might reflect poorly on the fact that both of the terms represent the same factor xy scaled by some constants. So in this experiment we propose an alternative UDE architecture, defined as follows:

$$\begin{aligned} \dot{x} &= \alpha x - U(x, y; \theta) \\ \dot{y} &= -\beta y + \varphi U(x, y; \theta) \end{aligned} \tag{2.12}$$

Where U is a single-output neural network and φ is a learnable parameter. Then we trained the UDE in the same manner as previously and used the SINDy algorithm to recover the equations.

In the end, the UDE yielded a final loss of 1.186×10^{-4} and the UDE-enhanced method recovered the following equations:

$$\begin{aligned} \dot{x} &= 1.256x - 0.815xy \\ \dot{y} &= -1.729 + 0.506xy \end{aligned} \tag{2.13}$$

By comparing it with the previous architecture (fig 2.3.14) the alternative UDE architecture is slightly better in extrapolating the dynamics from the small training set, albeit it still requires post-identification parameter estimation in order to fix the parameters.

EXPERIMENT 3 (LOTKA-VOLTERRA WITH OTHER PARAMETERS) In this experiment, we will follow the same pipeline as described in the first experiment. However, we will change the parameters of the Lotka-Volterra equations, in particular we will define the following Lotka-Volterra equations:

$$\begin{aligned} \dot{x} &= 2x - xy \\ \dot{y} &= cxy - 3y \\ (x_0, y_0) &= (0.4, 4) \end{aligned} \tag{2.14}$$

Where we will set $c = 0.5, 2$. Also, the time range from which we obtain the training set is $t \in [0, 2]$. The objective of this experiment is to test the UDE-enhanced SINDy method even further and attempt to see if there is any "preference" for any scaling factor of c.

Tables 2.3.4, 2.3.5 report the recovered coefficients of the original SINDy method and the enhanced SINDy method. As we can see, this time the original SINDy method managed to recover the coefficients without introducing unnecessary terms; moreover, it also inferred the growth rates of x, y accurately. However, in both cases the guessed c coefficient was inaccurate, while the UDE-enhanced method managed to obtain a closer guess.

In a certain sense, we have a trade-off between the two SINDy models: in the original case, we have an accurate estimate of the growth rates but an inaccurate estimate of the interaction coefficient. On the other hand, with the UDE-enhanced model we have a more accurate estimate of the interaction term, but a slightly more inaccurate estimate of the growth rates. To know which of the two is more preferable, we will simulate the recovered models and compare their errors with respect to the original trajectory.

In fact, from figures 2.3.15 and 2.3.16 we can see that the recovered equations with the UDE-enhanced SINDy methods are able to extrapolate the dynamics of the Lotka-Volterra equations much better than the original SINDy method. In fact, in the graph of the first case (fig. 2.3.15) the average of absolute errors between the simulation reconstructed with the UDE-enhanced method and the original equations tends to stay below the order of 10^0, while the

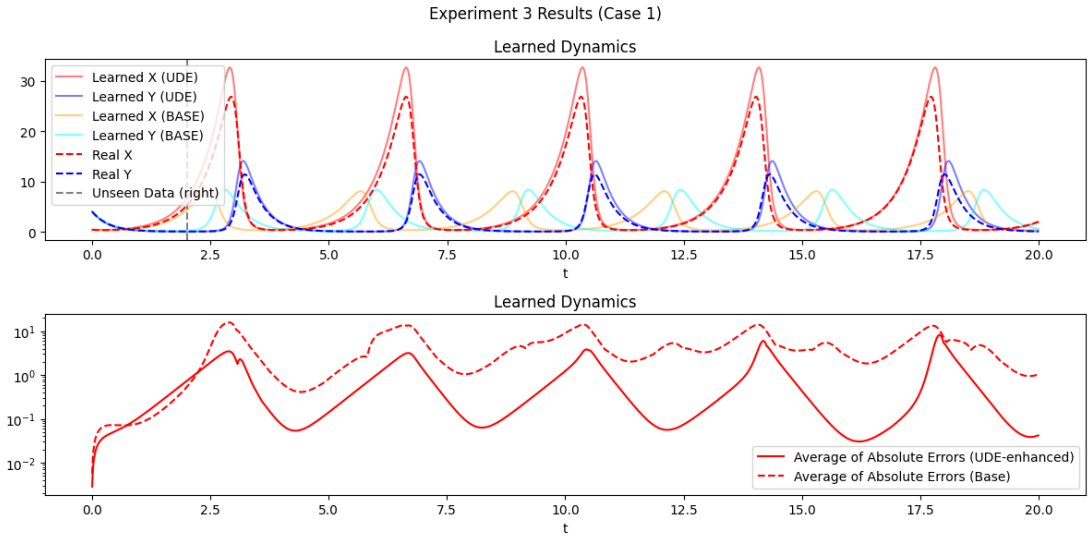

Figure 2.3.15: Simulation of the recovered equations, case 1 ($c = 0.5$)

other method stays above 10^0.

In the end, we can conclude that indeed UDEs provide a way to enhance the SINDy algorithm through the partial knowledge of the original dynamics.

Name	**Recovered** $\dot{x}$	**Recovered** $\dot{y}$
Original LV equations	$2x - xy$	$-3y + 0.5xy$
Original SINDy	$2.002x - 0.997xy$	$-3.007y + 1.256xy$
UDE-enhanced SINDy	$1.993x - 0.836xy$	$-3.053y + 0.426xy$

Table 2.3.4: Recovered coefficients for $c = 0.5$

Name	**Recovered** $\dot{x}$	**Recovered** $\dot{y}$
Original LV equations	$2x - xy$	$-3y + 2xy$
Original SINDy	$2.002x - 0.997xy$	$-3.007y + 1.256xy$
UDE-enhanced SINDy	$1.998x - 0.942xy$	$-2.939y + 1.801xy$

Table 2.3.5: Recovered coefficients for $c = 2$

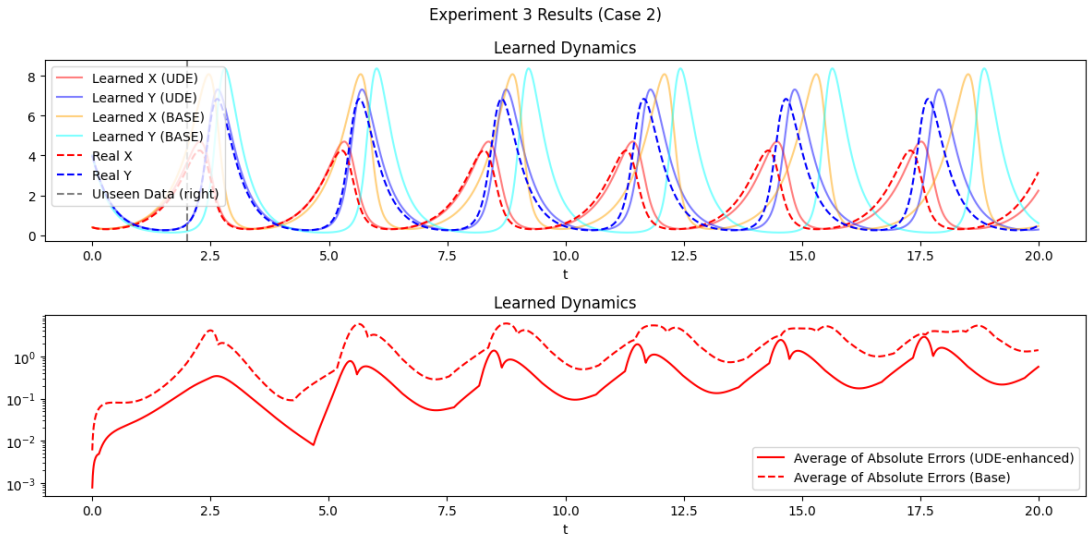

Figure 2.3.16: Simulation of the recovered equations, case 2 ($c = 2$)

Chapter 3

Advanced Case Study I: Neuron Modeling

3.1 Single Firing-Rate Neurons

3.2 Network of Firing-Rate Neurons

Chapter 4

Advanced Case Study II: Chua's Circuit Model

4.1 ???

Chapter 5

Concluding Remarks

Bibliography

[1] F. Rosenblatt. "The perceptron: A probabilistic model for information storage and organization in the brain." In: *Psychological Review* 65.6 (1958), pp. 386–408. ISSN: 0033-295X. DOI: 10.1037/h0042519. URL: http://dx.doi.org/10.1037/h0042519.

[2] D. Ludwig, D. D. Jones, and C. S. Holling. "Qualitative Analysis of Insect Outbreak Systems: The Spruce Budworm and Forest". In: *The Journal of Animal Ecology* 47.1 (Feb. 1978), p. 315. ISSN: 0021-8790. DOI: 10.2307/3939. URL: http://dx.doi.org/10.2307/3939.

[3] G. Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of Control, Signals and Systems* 2.4 (Dec. 1989), pp. 303–314. ISSN: 1435-568X. DOI: 10.1007/BF02551274. URL: https://doi.org/10.1007/BF02551274.

[4] Robert Tibshirani. "Regression Shrinkage and Selection Via the Lasso". In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 58.1 (Jan. 1996), pp. 267–288. ISSN: 1467-9868. DOI: 10.1111/j.2517-6161.1996.tb02080.x. URL: http://dx.doi.org/10.1111/j.2517-6161.1996.tb02080.x.

[5] James D Murray. *Mathematical Biology*. en. Ed. by J D Murray. 3rd ed. Interdisciplinary Applied Mathematics. New York, NY: Springer, Jan. 2002.

[6] Kevin P Murphy. *Machine learning: a probabilistic perspective*. Cambridge, MA, 2012.

[7] G. Teschl. *Ordinary Differential Equations and Dynamical Systems*. Graduate studies in mathematics. American Mathematical Society, 2012. ISBN: 9780821891056. URL: https://books.google.it/books?id=FSObYfuWceMC.

[8] *Neural Fields: Theory and Applications*. Springer Berlin Heidelberg, 2014. ISBN: 9783642545931. DOI: 10.1007/978-3-642-54593-1. URL: http://dx.doi.org/10.1007/978-3-642-54593-1.

[9] Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems". en. In: *Proc. Natl. Acad. Sci. U. S. A.* 113.15 (Apr. 2016), pp. 3932–3937.

[10] Ricky T. Q. Chen. *torchdiffeq*. 2018. URL: https://github.com/rtqichen/torchdiffeq.

[11] Ricky T. Q. Chen et al. "Neural Ordinary Differential Equations". In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018. URL: https://proceedings.neurips.cc/paper_files/paper/2018/file/69386f6bb1dfed68692a24c8686939b9-Paper.pdf.

[12] S.H. Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. CRC Press, 2018. ISBN: 978-0-429-96111-3. URL: https://books.google.it/books?id=A0paDwAAQBAJ.

[13] Christopher M. Bishop. *Pattern recognition and machine learning*. Information Science and Statistics. New York, NY: Springer Science+Business Media, LLC, 2019. 758 pp. ISBN: 978-0-387-31073-2.

[14] M. Raissi, P. Perdikaris, and G.E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". en. In: *Journal of Computational Physics* 378 (Feb. 2019), pp. 686–707. ISSN: 00219991. DOI: 10.1016/j.jcp.2018.10.045. URL: https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125.

[15] Kathleen Champion et al. "A Unified Sparse Optimization Framework to Learn Parsimonious Physics-Informed Models From Data". In: *IEEE Access* 8 (2020), pp. 169259–169271. DOI: 10.1109/ACCESS.2020.3023625.

[16] Christopher Rackauckas et al. *Universal Differential Equations for Scientific Machine Learning*. 2020. arXiv: 2001.04385 [cs.LG].

[17] Brian de Silva et al. "PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data". In: *Journal of Open Source Software* 5.49 (2020), p. 2104. DOI: 10.21105/joss.02104. URL: https://doi.org/10.21105/joss.02104.

[18] George Em Karniadakis et al. "Physics-informed machine learning". In: *Nature Reviews Physics* 3.6 (May 2021), pp. 422–440. ISSN: 2522-5820. DOI: 10.1038/s42254-021-00314-5. URL: http://dx.doi.org/10.1038/s42254-021-00314-5.

[19] Nicolas Brunel and Vincent Hakim. "Neuronal Dynamics". In: *Statistical and Nonlinear Physics*. Ed. by Bulbul Chakraborty. New York, NY: Springer US, 2022, pp. 495–516. ISBN: 978-1-0716-1454-9. DOI: 10.1007/978-1-0716-1454-9_359. URL: https://doi.org/10.1007/978-1-0716-1454-9_359.

[20] Alan A. Kaptanoglu et al. "PySINDy: A comprehensive Python package for robust sparse system identification". In: *Journal of Open Source Software* 7.69 (2022), p. 3994. DOI: 10.21105/joss.03994. URL: https://doi.org/10.21105/joss.03994.

[21] Philip K. Shiu et al. "A Drosophila computational brain model reveals sensorimotor processing". en. In: *Nature* 634.8032 (Oct. 2024), pp. 210–219. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07763-9. URL: https://www.nature.com/articles/s41586-024-07763-9.

[22] Rasha M. Yaseen et al. "Effect of the Fear Factor and Prey Refuge in an Asymmetric Predator–Prey Model". In: *Brazilian Journal of Physics* 54.6 (Sept. 2024). ISSN: 1678-4448. DOI: 10.1007/s13538-024-01594-9. URL: http://dx.doi.org/10.1007/s13538-024-01594-9.